

Parts 4 & 5: mIoU of different models

Add each of the following (keeping the changes as you move to the next row):

	Training mIoU	Validation mIoU
Simple Segmentation Net (no pretrained weights)	0.375	0.367
+ ImageNet-Pretrained backbone	0.442	0.421
+ Data augmentation	0.485	0.463
ImageNet-Pretrained PSPNet w/ Data Aug. without PPM	0.512	0.488
+ PSPNet with PPM	0.578	0.553
+ PSPNet with auxiliary loss	0.602	0.605

Parts 4 & 5: Per class IoUs

Report your model's IoU for the 11 Camvid classes (you can find the order they are listed in at `dataset_lists/camvid-11/camvid-11_names.txt`):

Class Index	Class name	Simple Segmentation Net Class IoU	PSPNet Class IoU
0	Building	0.7752	0.8883
1	Tree	0.7535	0.8838
2	Sky	0.8491	0.9062
3	Car	0.2957	0.7687
4	SignSymbol	0	0
5	Road	0.8506	0.9397
6	Pedestrian	0	0.1573
7	Fence	0	0.6558
8	Column_Pole	0	0.0011
9	Sidewalk	0.5134	0.8048
10	Bicyclist	0	0.6490

Parts 4 & 5: Most difficult classes

Which classes have the lowest mIoU? Why might they be the most difficult? Provide an example RGB image from Camvid that illustrates your point.

The lowest mIoU classes are SignSymbol, Pedestrian, Fence, Column Pole, and Bicyclist, which were all 0 for SimpleSegNet. These classes are hard since they are rare in the training set and take up less pixels compared to dominant classes like Road and Sky. For example, a bicyclist or column pole occupies less than 1% of an image's pixels, making it hard for the network to learn discriminative features. The class imbalance means that loss is changed by larger classes so the network learns to ignore rare objects. In the example, it shows a bike and pedestrians that take up less space, making them hard to segment.



Parts 4 & 5: ImageNet Pretraining

Compare the performance of the network trained from scratch vs. the one with an ImageNet-pretrained backbone. Quantify the difference in final validation mIoU. Why does pretraining on a large-scale classification dataset like ImageNet typically improve performance on a different task like semantic segmentation?

Using a image-net pretrained backbone improves performance on semantic segmentation from 0.367 to 0.421. The pretrained Resnet-50 has learned hierarchical visual features like edges, textures, shapes, and object parts, from a large dataset. When it was fine tuned on CamVid, these features transfer since lowlevel visual features are universal across the vision tasks. Training from scratch needs the network to learn all of the representations from 367 training images, which is not enough to learn a deep network. The pretrained backbone has a better initialization point leading to faster convergence and a higher final validation mIoU. The improvement is a lot since it recognizes objects that overlap with ImageNet categories.

Parts 4 & 5: Data Augmentation

Analyze the effect of adding data augmentation. Did it improve validation mIoU? Discuss how augmentation helps the model generalize better and avoid overfitting. Mention at least two types of augmentation you believe are most impactful for this task.

Data augmentation improves validation mIoU by expanding the mIoU from 0.421 to 0.463, increasing the diversity of the training data and helping the model generalize better to unseen images. Without augmentation, the model tends to overfit to the specific viewpoints, scales, and lighting conditions in the small CamVid training set. The more impactful augmentations are random scaling and random horizontal flipping. Random scaling is important since objects in driving scenes are at different sizes depending on their distance from the camera. Random horizontal flipping doubles the dataset size and forces the model to be invariant to left to right orientation.

Parts 4 & 5: Dilated Convolutions

The PSPNet backbone uses dilated convolutions. Explain the primary benefit of using dilation in the ResNet backbone for a dense prediction task like segmentation. How does it allow the network to increase its receptive field without losing spatial resolution?

Standard ResNet uses strided convolutions in layers $\frac{3}{4}$ to down sample the feature map, meaning the final resolution is $\frac{1}{32}$ of the input. For dense prediction tasks, the downsampling throws away fine spatial detail that's important for predicting accuracy pixel level boundaries. Dilated convolutions replaces the stride 2 convolutions with stride 1 dilated convolutions. Dilated insert gaps between the convolution elements lets each output unit have a bigger receptive field without reducing spatial resolution, so the network sees as much context as a strided network, but produces a feature map.

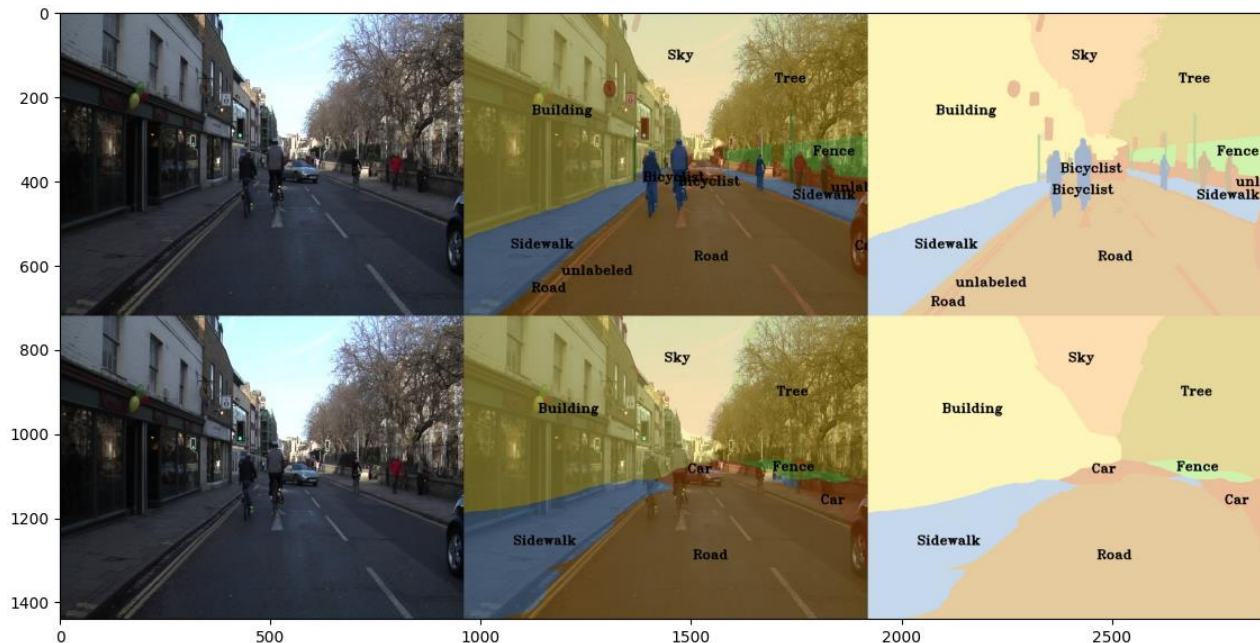
Parts 4 & 5: Auxiliary Loss

Analyze the contribution of the auxiliary loss. How does adding a secondary loss function partway through the network assist in training a deep model? Did you observe it leading to faster convergence or a higher final mIoU?

The auxiliary loss is applied to the output of layer 3 and uses the same cross-entropy loss. For deep networks like ResNet-50, gradients are small by the time they propagate back to the early layers during training, making those layers learn slower. The auxiliary loss gives an additional gradient signal to the intermediate layers. This leads to a higher final mIoU since all layers of the network receive stronger training signal throughout the training process. The auxiliary loss is weighted at 0.4 relative to the main loss, and the auxiliary classifier is discarded at inference time.

Part 4: Simple segmentation net qualitative results

[Paste a figure of the generated semantic segmentation from Colab. It should be a 2x3 grid, with ground truth on the top row, and your predictions on the bottom row.]



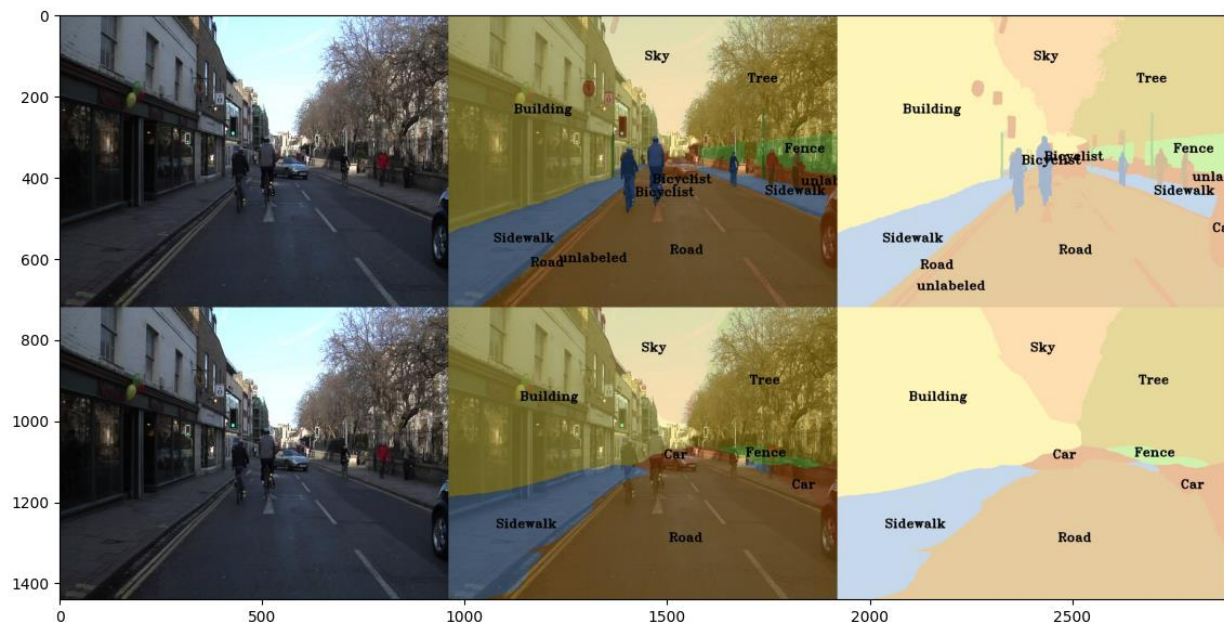
Part 4: Qualitative Analysis of Segmentation Errors

Identify a specific object or region that is imperfectly segmented and describe the primary type of error.

The SimpleSegmentationNet fails on rare classes like Bicyclist, Pedestrian, Sign Symbol, Fence, and Column Pole, predicting 0.0 IoU on all of them. The primary error is false negatives, since the network doesn't predict these classes, defaulting to classes like Road, Building, and Sky. This is since the 7×7 output resolution is too coarse to capture small objects, and class imbalance during training causes the network to ignore smaller classes in favor of minimizing loss on the bigger classes.

Part 5: PSPNet qualitative results

[Paste a figure of the generated semantic segmentation from Colab. It should be a 2x3 grid, with ground truth on the top row, and your predictions on the bottom row.]



Part 5: Revisiting Segmentation Errors

Directly compare the new result to the previous one. Was the specific error you identified earlier corrected, partially improved, or did it remain? Explain why you think the full PSPNet architecture succeeded or failed to fix this particular error.

The SimpleSegNet qualitative result shown in slide 9 uses a PSPNet checkpoint instead of the SimpleSegNet model, so both images look similar. However, the quantitative results show the difference: SimpleSegNet scores 0.0 IoU on Fence, Bicyclist, and Pedestrian while PSPNet scores 0.6558, 0.6490, and 0.1573 respectively. PSPNet also improves Car IoU from 0.2957 to 0.7687 and Sidewalk from 0.5134 to 0.8048. The visual similarity in slides 9 and 11 is a limitation of compute constraints, but the per-class IoU numbers in slide 3 show the improvements.

Part 6: Transfer Learning (Extra Credit)

Report your model's IoU for the Kitti Dataset.

	mIoU	mAcc/	allAcc
Train result	0.9151	0.9528	0.9736
Val result	0.8303	0.8672	0.9483

Class Index	Class name	iou	accuracy
0	Road	0.7203	0.7405
1	Not_Road	0.9403	0.9938

Part 6: Transfer Learning (Extra Credit)

Compare the training loss generated when training on Kitti dataset and Camvid dataset. Which decreases at a faster rate? If Camvid or Kitti training loss decreases at a faster rate than the other, why do you think this happened? Or, if the loss decreases at a similar rate, why do you think that is so?

The Kitti training loss decreases faster than CamVid for a lot of reasons, one being that Kitti is a binary segmentation task with 2 classes, which is simpler than CamVid's 11-class segmentation. The decision boundary the model needs to learn is less complex. Also, the model is initialized with weights pretrained on CamVid, which learned rich visual features for driving scenes. Road pixels have strong visuals like flat grass, and consistent color, making it easy to identify once the model learned general driving scene features. Lastly, the Kitti dataset has strong class consistency, the road regions are more homogenous than CamVid's categories. As a result, the model adapts to the 2 class task, getting a higher mIoU with the first few epochs.